

🎤 Présentation d'Avancement : Plateforme BMCI & Agent Vocal Temps Réel

✈️ Diapositive 1 : Page de Garde

Simulation de Client Bancaire par IA

Plateforme de Fine-Tuning BMCI & Agent Vocal Interactif Temps Réel

- **Auteur** : Stagiaire R&D - **BMCI**
 - **Objectif** : Concevoir une plateforme d'entraînement et d'évaluation des modèles pour le rôle de client bancaire en colère.
 - **Architecture** : Modèles locaux fine-tunés, Interface Streamlit, LiveKit WebRTC.
-

✈️ Diapositive 2 : Le Cas d'Usage de la Simulation

Incarner le Client Mécontent (M. Orens)

- **Le Scénario** : Retrait urgent de **100 000 dirhams** pour l'achat d'une maison aujourd'hui avant midi. Le conseiller annonce la limite de retrait de **50 000 DH**.
 - **Exigences Métier & Techniques** :
 - Voix transmettant la colère et l'impatience.
 - Barge-in : L'agent doit s'interrompre dès que l'utilisateur lui parle.
 - Latence basse (idéalement < 1,5s).
-

✈️ Diapositive 3 : Phase 1 - Dataset & Fine-Tuning Local

Adapter les LLMs au Métier Bancaire

- **Création d'un Dataset Fictif BMCI** :
 - Échanges de négociation sur 9 cas : *carte bloquée, virement en retard, appli bloquée, frais bancaires, chèque, crédit, etc.*
 - **Fine-Tuning LoRA (Low-Rank Adaptation)** :
 - Modèles testés : *Qwen, SmolLM2, TinyLlama, BloomZ, CroissantLLM.*
 - **Modèle local le plus exploitable** : `Qwen2.5-0.5B-BMCI-Client-Finetune-1.`
-

✈️ Diapositive 4 : Phase 1 - Difficultés du Fine-Tuning Local

Contraintes Techniques & Matérielles

- **Respect du rôle de client** : Tendance naturelle des LLMs à répondre comme conseiller → Résolu par des prompts système stricts, du few-shot et des filtres post-génération.
 - **Dérives de contexte** : Perte du sujet de départ (ex: parler de crédit au lieu de virement) → Nécessite d'enrichir le dataset d'exemples de recentrage.
 - **Absence de GPU local (CUDA Windows)** : Infécond d'entraîner sur CPU → Bascule des calculs d'entraînement sur Colab / Kaggle et blocage local des modèles lourds (> 3B).
 - **Checkpoints et Dépendances** : Crashes de disques Colab et conflits de packages (*transformers*, *peft*, *torch*) → Logic de backup automatique sur Drive et scripts robustes.
-

✈ Diapositive 5 : Phase 1 - Streamlit & Framework d'Évaluation

Mesurer les Progrès Objectivement

- **Adaptation de l'application Streamlit** :
 - Chargement des modèles locaux fine-tunés et des formats .json, .jsonl, .csv.
 - Onglets "Chat Client BMCI" et "Évaluation".
 - Garde-fous intégrés contre les sorties de rôle.
 - **Framework d'Évaluation Avancé** :
 - Scénarios multi-modèles (Promptfoo) + Red-Teaming (attaques vocales).
 - Scoring automatique G-Eval (LLM-as-a-judge) + Proxy RAGAS (Fidélité).
 - Métriques : *respect du rôle, pertinence, sécurité, claims non supportés*.
-

✈ Diapositive 6 : Phase 2 - Benchmark STT (Reconnaissance Vocale)

Modèle Réussi vs. Tests Non Concluants

- ● **Réussite : Cohere Transcribe v2 (Cloud)** :
 - Taux d'erreur de mots (WER) très bas (5.82%), ultra-rapide en streaming, très robuste aux bruits de fond d'agence.
 - ● **Non Concluants / Échecs** :
 - **ElevenLabs Scribe v2 (Cloud)** : Précision excellente (3.12% de WER), mais latence de 21s (inutilisable en direct).
 - **Whisper Large-Turbo (Local)** : Trop lourd pour le CPU local (132s de calcul pour transcrire 3 minutes d'audio).
-

✈ Diapositive 7 : Phase 2 - Benchmark TTS (Synthèse Vocale)

Modèle Réussi vs. Tests Non Concluants

- **Réussite : Mistral Voxtral (Voix : Marie angry) :**
 - Colère native très convaincante, latence faible (1.57s) et quotas d'API stables.
 - **Non Concluants / Échecs :**
 - **Hume AI (Cloud) :** Voix fantastique (MOS 4.03) mais bloquée par des quotas stricts (erreurs 429).
 - **F5-TTS (Local) :** Infécond en temps réel sur CPU (128s de calcul pour 3s de voix).
 - **Kokoro & MeloTTS (Locaux) :** Voix françaises trop robotiques, plates, et sans expressivité émotionnelle.
-

✈ Diapositive 8 : Phase 3 - Agent Vocal Temps Réel (LiveKit)

Une Expérience Fluide et Résiliente

- **Les Atouts de LiveKit :**
 - *Full-Duplex & Barge-in* : Conversation continue avec interruptions millisecondes dès que l'utilisateur prend la parole.
 - **Optimisations Appliquées :**
 - *Superviseur de Résilience* (`run_agent.py`) : Auto-restart en moins de 2s lors des déconnexions WebRTC.
 - *Gestion fine de la parole* (`min_delay=0.8`) : Attente minimale pour laisser le temps au STT de finaliser la transcription.
 - *Filtre anti-didacalies* : Nettoyage regex automatique des crochets (`[gasp]`) et majuscules pour éviter que la voix ne les épelle.
-

✈ Diapositive 9 : L'Avenir : Analyse de l'Option Speech-to-Speech (Moshi)

6 Verrous Techniques Majeurs Identifiés

1. **Infrastructure GPU lourde** : VRAM minimale de 16-24 Go requise, coût cloud mensuel fixe élevé (NVIDIA A100/H100).
2. **Couche de transport audio complexe** : Nécessité de développer l'interfaçage WebSocket et la gestion de la gigue réseau (Jitter Buffer).
3. **VAD & Interruption complexe** : Obligation de coder des purges instantanées de buffers pour couper le flux de l'IA.
4. **Pas de support Windows natif** : Obligation d'utiliser WSL2 (complexité de partage GPU, latence audio).
5. **Pas de passerelle VoIP/SIP** : Nécessité de ponts complexes pour relier Moshi au réseau téléphonique.
6. **Pas de Function Calling** : Obligation d'analyser le monologue intérieur de Moshi pour

✈ Diapositive 10 : Bilan & Perspectives

Où en sommes-nous et où allons-nous ?

- **État Actuel** : Dataset BMCI prêt, modèle Qwen fine-tuné et testé en local, interface Streamlit de chat et d'évaluation stable, agent vocal fonctionnel sur LiveKit Cloud.
- **Prochaines Étapes** :
 1. Enrichir le dataset de fine-tuning (sécurité, résistance aux pièges, respect du contexte).
 2. Stabiliser les entraînements de modèles sur Kaggle/Colab.
 3. Étudier la viabilité économique d'un serveur cloud GPU pour basculer sur des modèles natifs Speech-to-Speech.